stat_close.py — 21.4절: 확률 명세의 E 층과 S 층(큰수의 법칙·비율 편차·DKW)

```
"""stat_close: 확률 명세의 오라클 — 큰수의 법칙과 비율 편차(21.5절).

확률 명세는 "출력의 기댓값이 μ*이다"(몬테카를로 π: E[4·1{x²+y² ≤ 1}] = π, 허친슨: E[zᵀAz] = tr A) 또는
"출력의 분포가 F*이다"(난수: 균등분포)의 꼴이다.  값 하나로는 맞고 틀림이 정해지지 않으므로 두 층으로 나눈다.

  E 층(정확)   주어진 표본에서 코드의 값 = 의도한 식의 값.  대수 식이면 GF로 모든 표본값에서(20.2절),
               순서가 들어가면 유리수(순서가 있는 정확 산술)로 실제 표본에서.
  S 층(통계)   표본이 독립·동일 분포이고 X_i ∈ [a, b]이면, 어느 때나 유효한 비율 편차 판정:
                 N_k = N_0·2^k,  δ_k = δ / ((k+1)(k+2))  (Σ δ_k = δ),
                 |X̄_{N_k} - μ*| > (b - a) √(ln(2/δ_k) / (2 N_k))  이면 반례(유의수준 δ).
               (i) 명세가 참이면 끝까지 한 번이라도 반례를 낼 확률 <= δ   (호에프딩 + 합집합 상계)
               (ii) 기댓값이 μ ≠ μ*이면 강한 큰수의 법칙으로 X̄ -> μ, 문턱 -> 0 이므로 확률 1로 언젠가 반례.
                    대략 N >= 2 (b-a)² ln(2/δ_k) / (μ - μ*)² 에서 잡힌다.
               분포 명세는 DKW 부등식  P(sup|F_N - F*| > ε) <= 2 exp(-2Nε²)  를 같은 일정으로 쓴다.
"""
import math, random
from fractions import Fraction as Fr
import numpy as np
import oracle_close as oc
from oracle_close import GF, gf_rand, ValueBranch

DELTA = 1e-6


def thr(N, k, width, delta=DELTA):
    return width * math.sqrt(math.log(2 / (delta / ((k + 1) * (k + 2)))) / (2 * N))


def anytime_mean(stat, target, width, N0=1 << 10, K=10, delta=DELTA):
    """stat(N) -> 표본 평균(앞 N개).  반환 dict(verdict, N, est, thr)."""
    for k in range(K + 1):
        N = N0 << k; est = stat(N); t = thr(N, k, width, delta)
        if not np.isfinite(est) or abs(est - target) > t:
            return dict(verdict="반례", N=N, est=est, thr=t, k=k)
    return dict(verdict="δ-증명", N=N, est=est, thr=t, k=K)


def anytime_dkw(sample, N0=1 << 10, K=8, delta=DELTA):
    """sample(N) -> [0,1) 위 값 N개.  균등분포 명세의 DKW 판정."""
    for k in range(K + 1):
        N = N0 << k; x = np.sort(np.asarray(sample(N), float))
        if len(x) != N or not np.all(np.isfinite(x)): return dict(verdict="반례", N=N, D=float("nan"), thr=0)
        D = max(np.max(np.arange(1, N + 1) / N - x), np.max(x - np.arange(N) / N))
        eps = math.sqrt(math.log(2 / (delta / ((k + 1) * (k + 2)))) / (2 * N))
        if D > eps: return dict(verdict="반례", N=N, D=float(D), thr=eps, k=k)
    return dict(verdict="δ-증명", N=N, D=float(D), thr=eps, k=K)


# ---------------------------------------------------------------- 몬테카를로 π
def mc_layers(src):
    fn_f = oc.float_version(src, "mc_pi", {})
    # E 층: 유리수 표본(배정밀도 난수는 이분 유리수라 정확히 옮겨진다)에서 코드의 값 = 4·#{x²+y² <= 1}/N
    fn_q, g = oc.exact_version(src, "mc_pi", {})
    rng = np.random.default_rng(11); pts = rng.random((512, 2))
    Q = [(Fr(float(a)), Fr(float(b))) for a, b in pts]
    try:
        out = fn_q(Q); want = Fr(4 * sum(1 for a, b in Q if a * a + b * b <= 1), len(Q))
        E = "통과" if Fr(out) == want else f"반례: {float(out):.6f} != {float(want):.6f}"
    except Exception as e:
        E = f"반례: 실행 실패 {type(e).__name__}"
    # S 층: 독립 균등 표본, 어느 때나 유효한 비율 편차
    big = np.random.default_rng(7).random((1 << 20, 2))
    def stat(N):
        try: return float(fn_f(big[:N].tolist()))
        except Exception: return float("nan")
    S = anytime_mean(stat, math.pi, 4.0)
    return E, S


# ---------------------------------------------------------------- 허친슨 대각합 추정
HUTCH = '''
def hutch(A, zs):
    n = len(A)
    s = 0
    for z in zs:
```

```
        Az = [0] * n
        for i in range(n):
            for j in range(n):
                Az[i] = Az[i] + A[i][j] * z[j]
        q = 0
        for i in range(n):
            q = q + z[i] * Az[i]
        s = s + q
    return s / len(zs)
'''


def hutch_layers(src):
    # E 층: GF, 모든 A와 모든 z에서  출력 = (1/m) Σ zᵀ A z   (대수 명세, 20.2절)
    fn, g = oc.exact_version(src, "hutch", {})
    E = "통과"
    for sd in range(3):
        rng = random.Random(sd); n, m = 3 + sd, 4 + sd
        A = [[gf_rand(rng) for _ in range(n)] for _ in range(n)]; zs = [[gf_rand(rng) for _ in range(n)] for _ in range(m)]
        try:
            out = fn(A, zs)
            want = sum((sum((zs[t][i] * A[i][j] * zs[t][j] for i in range(n) for j in range(n)), GF(0)) for t in range(m)), GF(0)) / GF(m)
            if GF.c(out).v != want.v: E = f"반례(n={n}, m={m})"; break
        except ValueBranch as e:
            E = f"범위 밖: {e}"; break
        except Exception as e:
            E = f"반례: 실행 실패 {type(e).__name__}"; break
    # S 층: 라데마허 z, zᵀAz ∈ [-Σ|a|, Σ|a|]
    ff = oc.float_version(src, "hutch", {}); rng = np.random.default_rng(5)
    A = rng.standard_normal((6, 6)); A = (A + A.T) / 2; W = 2 * np.abs(A).sum()
    Z = rng.choice([-1.0, 1.0], size=(1 << 16, 6))
    def stat(N):
        try: return float(ff(A.tolist(), Z[:N].tolist()))
        except Exception: return float("nan")
    S = anytime_mean(stat, float(np.trace(A)), W, N0=1 << 8, K=8)
    return E, S


# ---------------------------------------------------------------- 선형 합동 난수의 균등성
def lcg_layers(src):
    import domains as Dm
    fn = oc.float_version(src, "lcg", {})
    try:
        out = fn(12345, 200); E = "통과" if out == Dm.lcg_ref(12345, 200) else "반례(의도한 점화식과 다름)"
    except Exception as e:
        out = None; E = f"반례: 실행 실패 {type(e).__name__}"
    def sample(N):
        try: return [v / 2147483648 for v in fn(20261030, N)]
        except Exception: return [float("nan")] * N
    S = anytime_dkw(sample)
    return E, S
```

작동 확인 stat_test.py — 21.4절의 실험

```
"""21.5절 실험: 확률 명세를 E 층(정확)과 S 층(큰수의 법칙 + 비율 편차)으로 판정한다."""
import json, math, random, signal, time, warnings
import numpy as np
warnings.filterwarnings("ignore")
import stat_close as SC, domains as Dm, mutate


def _al(s, f): raise TimeoutError()

def run(key, src):
    signal.signal(signal.SIGALRM, _al); signal.alarm(60)
    try:
        if key == "mcpi": return SC.mc_layers(src)
        if key == "hutch": return SC.hutch_layers(src)
        return SC.lcg_layers(src)
    except TimeoutError:
        return "반례: 시간 초과", dict(verdict="반례", N=0)
    finally:
        signal.alarm(0)


```

```
def baseline(key, src):
    """그 분야의 흔한 시험 한 번(허용치 고정)."""
    try:
        if key == "mcpi":
            fn = SC.oc.float_version(src, "mc_pi", {}); pts = np.random.default_rng(3).random((20000, 2)).tolist()
            return not abs(fn(pts) - math.pi) < 0.05
        if key == "hutch":
            fn = SC.oc.float_version(src, "hutch", {}); rng = np.random.default_rng(9)
            A = rng.standard_normal((6, 6)); A = (A + A.T) / 2; Z = rng.choice([-1.0, 1.0], size=(4000, 6))
            return not abs(fn(A.tolist(), Z.tolist()) - np.trace(A)) < 0.5
        fn = SC.oc.float_version(src, "lcg", {}); x = np.array(fn(1, 10000)) / 2147483648
        return not (abs(x.mean() - 0.5) < 0.01 and 0 <= x.min() and x.max() < 1)
    except Exception:
        return True


def main():
    srcs = dict(mcpi=Dm.D["mcpi"]["src"], hutch=SC.HUTCH, lcg=Dm.D["lcg"]["src"])
    out = []
    for key, src in srcs.items():
        jobs = [("원본", "", src)] + [(k, w, s) for k, w, line, s in mutate.mutants(src)]
        for kind, where, s in jobs:
            t0 = time.time(); E, S = run(key, s)
            signal.alarm(20)
            try: b = baseline(key, s)
            except TimeoutError: b = True                # 끝나지 않는 실행은 기준선도 잡는다
            finally: signal.alarm(0)
            out.append(dict(key=key, kind=kind, where=where, src=s, E=E, S=S["verdict"], S_N=S.get("N"),
                            S_est=S.get("est", S.get("D")), S_thr=S.get("thr"), BR=b, sec=round(time.time() - t0, 2)))
        print(key, len(jobs), flush=True)
    json.dump(out, open("stat_results.json", "w"), ensure_ascii=False, indent=0, default=float)

    # 원본의 거짓 반례율: 독립 표본 200번
    fn = SC.oc.float_version(srcs["mcpi"], "mc_pi", {}); fa = 0
    for r in range(200):
        big = np.random.default_rng(1000 + r).random((1 << 16, 2))
        res = SC.anytime_mean(lambda N: float(fn(big[:N].tolist())), math.pi, 4.0, N0=1 << 10, K=6)
        fa += res["verdict"] == "반례"
    # 검출 크기: 반지름 제곱 1+ε인 치우친 추정량, 예측 N과 실제 N
    det = []
    for eps in (0.2, 0.05, 0.02, 0.01):
        from scipy.integrate import quad
        bias = 4 * quad(lambda x: min(1.0, math.sqrt(max(0.0, 1 + eps - x * x))), 0, 1, limit=200)[0] - math.pi   # 정확한 치우침
        big = np.random.default_rng(77).random((1 << 22, 2)); r2 = big[:, 0] ** 2 + big[:, 1] ** 2
        cs = np.cumsum(r2 <= 1 + eps)
        res = SC.anytime_mean(lambda N: 4 * cs[N - 1] / N, math.pi, 4.0, N0=1 << 10, K=12)
        pred = next((N for k in range(13) for N in [1024 << k] if SC.thr(N, k, 4.0) < abs(bias) / 2), None)
        det.append(dict(eps=eps, bias=bias, N=res["N"] if res["verdict"] == "반례" else None, pred=pred))
    json.dump(dict(false_alarm=fa, runs=200, det=det), open("stat_extra.json", "w"), ensure_ascii=False, indent=1)
    print("거짓 반례", fa, "/ 200", det)


if __name__ == "__main__":
    main()
```

작동 확인 realbug_test.py — 22.4절: 실제 버그

```
"""23절: 오픈소스 수치 코드에서 실제로 보고·수정된 버그. 고치기 전/후 코드를 저장소 이력에서 그대로 꺼내 판정한다.

  R1 findiff  c24b367 (2026-07)  비균일 격자 짝수 계 도함수의 스텐실이 한 점 모자라 정확도 acc -> acc-1
  R2 pyclaw   f522337e (2026-06) 움직이는 벽 경계에서 가장 바깥 유령 칸을 채우지 않음(off-by-one)
  R3 scikit-fem 24601138 (2020-01) 라메 상수 (lambda, mu)를 뒤바꿔 돌려줌
  R4 nodepy   15c1027 (2021-04)  SSP22·SSP33의 Shu-Osher 배열에 열이 하나 더(대조군: 걸음 계산에는 영향 없음)

오라클
  B0 그 분야의 흔한 시험(제조해 수렴 차수, 흔한 시험값, 저장된 회귀 기준해)
  B1 실행 검사: 사용자의 한 번 실행에서 주장의 관계식(반올림 허용치)
  B2 닫힘: 정확 산술 재실행(라이브러리의 실수 배열·선형 풀이는 체 위의 정확한 연산으로 읽는다) + 정수 추적
"""
```

```
import importlib.util, json, math, random, subprocess, sys, types, warnings
from fractions import Fraction as Fr
import numpy as np
warnings.filterwarnings("ignore")
import oracle_close as oc
from oracle_close import GF, gf_rand, Tracer, TInt

RB = "../realbugs"
REPOS = "../repos"


def load(path, name, inject=None):
    spec = importlib.util.spec_from_file_location(name, path); m = importlib.util.module_from_spec(spec)
    if inject: m.__dict__.update(inject)
    spec.loader.exec_module(m)
    if inject: m.__dict__.update(inject)
    return m


# ---------------------------------------------------------------- 라이브러리 호출을 정확한 연산으로 읽는 numpy 대리자
def gf_solve(A, b):
    A = [[GF.c(x) for x in row] for row in np.asarray(A, dtype=object)]; b = [GF.c(x) for x in np.asarray(b, dtype=object).ravel()]
    n = len(A)
    for k in range(n):
        piv = next(i for i in range(k, n) if A[i][k].v != 0)
        A[k], A[piv] = A[piv], A[k]; b[k], b[piv] = b[piv], b[k]
        for i in range(k + 1, n):
            f = A[i][k] / A[k][k]
            for j in range(k, n): A[i][j] = A[i][j] - f * A[k][j]
            b[i] = b[i] - f * b[k]
    x = [None] * n
    for i in range(n - 1, -1, -1):
        s = b[i]
        for j in range(i + 1, n): s = s - A[i][j] * x[j]
        x[i] = s / A[i][i]
    return np.array(x, dtype=object)


class _Linalg:
    solve = staticmethod(gf_solve)


class ExactNP(types.ModuleType):
    """np 자리: array(..., dtype=float)는 체 원소 배열로, linalg.solve는 정확한 소거로.  나머지는 numpy 그대로."""
    def __init__(s): super().__init__("np_exact"); s.linalg = _Linalg()
    def __getattr__(s, k): return getattr(np, k)
    @staticmethod
    def array(x, dtype=None, **kw):
        a = np.array(x, dtype=object)
        if a.size == 0 or all(isinstance(e, (int, np.integer)) and not isinstance(e, bool) for e in a.ravel()):
            return np.array(x, dtype=dtype) if dtype not in ("float", float) else np.vectorize(GF.c, otypes=[object])(a)
        return np.vectorize(GF.c, otypes=[object])(a)


# ---------------------------------------------------------------- R1 findiff
def findiff_moment_ok(mod, deriv, acc, coords, exact):
    """모든 마디에서 주장한 정확도의 모멘트 조건  Σ w_j (x_{k+o_j} - x_k)^m = m! δ_{m,deriv},  m < deriv + acc."""
    N = len(coords); bad = []
    for k in range(N):
        r = mod.coefficients_non_uni(deriv, acc, coords, k)
        w, o = r["coefficients"], r["offsets"]
        for m in range(deriv + acc):
            want = math.factorial(m) if m == deriv else 0
            if exact:
                s = sum((GF.c(w[j]) * (coords[k + int(o[j])] - coords[k]) ** m for j in range(len(w))), GF(0))
                ok = s.v == GF(want).v
            else:
                terms = [float(w[j]) * (coords[k + int(o[j])] - coords[k]) ** m for j in range(len(w))]
                ok = abs(sum(terms) - want) <= 1e-9 * (sum(abs(t) for t in terms) + 1)
            if not ok: bad.append((k, m)); break
    return bad


```

```
def r1():
    out = {}
    for ver in ("bug", "fix"):
        mf = load(f"{RB}/findiff_{ver}/coefs.py", f"coefs_{ver}")
        me = load(f"{RB}/findiff_{ver}/coefs.py", f"coefs_{ver}_x", inject=dict(np=ExactNP()))
        # B0: 제조해 — 균일 격자(흔한 시험)와 매끄러운 늘림 격자, f = sin(3x), 2계 도함수, acc = 2
        def order(grid):
            errs = []
            for n in (41, 81, 161):
                s = np.linspace(0, 1, n); x = s if grid == "uniform" else s + 0.1 * np.sin(np.pi * s) / np.pi
                f = np.sin(3 * x); d2 = -9 * np.sin(3 * x); e = 0
                for k in range(n):
                    r = mf.coefficients_non_uni(2, 2, x, k)
                    e = max(e, abs(sum(r["coefficients"][j] * f[k + r["offsets"][j]] for j in range(len(r["offsets"]))) - d2[k]))
                errs.append(e)
            return math.log2(errs[-2] / errs[-1])
        p_uni, p_str = order("uniform"), order("stretched")
        # B1: 사용자의 한 비균일 격자(21마디)에서 모멘트 조건(반올림 허용치)
        xs = np.sort(np.random.default_rng(4).uniform(0, 1, 21)); b1 = {(d, a): findiff_moment_ok(mf, d, a, xs, False) for d in (1, 2, 3, 4) for a in (2, 4)}
        # B2: 모든 격자 — GF 무작위 비균일 격자와 GF 균일 격자, 모든 마디
        rng = random.Random(1)
        xg = np.array([gf_rand(rng) for _ in range(16)], dtype=object)
        x0, h = gf_rand(rng), gf_rand(rng); xu = np.array([x0 + h * i for i in range(16)], dtype=object)
        b2 = {(d, a): findiff_moment_ok(me, d, a, xg, True) for d in (1, 2, 3, 4) for a in (2, 4, 6)}
        b2u = {(d, a): findiff_moment_ok(me, d, a, xu, True) for d in (1, 2, 3, 4) for a in (2, 4, 6)}
        out[ver] = dict(
            B0=f"균일 격자 관측 차수 {p_uni:.2f}, 늘림 격자 {p_str:.2f} (공칭 2)",
            B0_detect_uniform=p_uni < 1.7, B0_detect_stretched=p_str < 1.7,
            B1=[f"d={d},acc={a}: 마디 {v[0][0]}, m={v[0][1]}" for (d, a), v in b1.items() if v],
            B2=[f"d={d},acc={a}: 마디 {v[0][0]}, 모멘트 m={v[0][1]}" for (d, a), v in b2.items() if v],
            B2_uniform=[f"d={d},acc={a}" for (d, a), v in b2u.items() if v])
    return out


# ---------------------------------------------------------------- R2 pyclaw
def pyclaw_src(rev):
    s = subprocess.run(["git", "-C", f"{REPOS}/pyclaw", "show", f"{rev}:examples/stegoton_1d/stegoton.py"], capture_output=True, text=True).stdout
    i = s.index("def moving_wall_bc"); j = s.index("\ndef ", i + 10)
    return s[i:j]


class _Dim:
    on_lower_boundary = True; on_upper_boundary = False


def wall_spec(qin, aux, ng, vwall):
    """움직이는 벽의 의도: 성분 0은 첫 내부 칸 값, 성분 1은 모든 유령 칸이 거울 칸의 반사  q1[ng-1-k] = 2·vwall·aux1[k] - q1[ng+k]."""
    q = qin.copy()
    for k in range(ng):
        q[0, k] = qin[0, ng]
        q[1, ng - 1 - k] = 2 * vwall * aux[1, k] - qin[1, ng + k]
    return q


def r2():
    out = {}
    pd = dict(t1=10.0, tw1=10.0, a1=0.1, trtime=1e9, trdone=False)
    for ver, rev in (("bug", "f522337e^"), ("fix", "f522337e")):
        src = pyclaw_src(rev)
        # B1: 사용자의 실행 그대로(유령 칸 2개, 부동소수), 경계 채움 뒤 의도한 관계를 반올림 허용치로
        g = dict(np=np); exec(src, g); fn = g["moving_wall_bc"]
        ng = 2; n = 20; rng = np.random.default_rng(0)
        st = types.SimpleNamespace(t=12.0, problem_data=pd, aux=rng.uniform(0.5, 2, (2, n)))
        qin = rng.standard_normal((2, n + 2 * ng)); qbc = qin.copy(); fn(st, _Dim(), 12.0, qbc, None, ng)
        t0 = (12.0 - 10.0) / 10.0; vw = -0.1 * (1 + math.cos(t0 * math.pi))
        want = wall_spec(qin, st.aux, ng, vw); b1 = np.max(np.abs(qbc - want)) > 1e-12
        # B2: 모든 유령 칸 수(추적 정수)와 모든 값(GF) — 대표 크기로
        fe, ge = oc.exact_version(src, "moving_wall_bc", dict(np=np))
        def run(ngh, seed, tr):
            rg = random.Random(seed); tr = tr or Tracer(); nn = 6
            aux = np.array([[gf_rand(rg) for _ in range(nn)] for _ in range(2)], dtype=object)
            qi = np.array([[gf_rand(rg) for _ in range(nn + 2 * ngh)] for _ in range(2)], dtype=object)
            q = qi.copy(); s_ = types.SimpleNamespace(t=12.0, problem_data=pd, aux=aux)
            old = oc._patched(ge, tr)
```

```
            try: fe(s_, _Dim(), 12.0, q, None, TInt(ngh, tr, {"num_ghost"}))
            except Exception as e: return False, dict(n=ngh, why=f"실행 실패 {type(e).__name__}")
            finally: oc._restore(ge, old)
            w_ = wall_spec(qi, aux, ngh, vw)
            for i in range(2):
                for j in range(nn + 2 * ngh):
                    if GF.c(q[i, j]).v != GF.c(w_[i, j]).v: return False, dict(n=ngh, why=f"성분 {i}, 칸 {j}")
            return True, dict(n=ngh)
        import domains as Dm
        b2 = Dm.close_sized(run, 1)
        out[ver] = dict(B0="저장된 회귀 기준해(expected_sols.npy)는 고치기 전 코드로 만든 것: 고친 뒤 다시 생성(커밋 메시지)",
                        B0_detect=(ver == "fix"), B1=bool(b1), B2=b2["verdict"], B2_info=str(b2.get("info", b2.get("meta", "")))[:80])
    return out


# ---------------------------------------------------------------- R3 scikit-fem
def r3():
    out = {}
    for ver, rev in (("bug", "24601138^"), ("fix", "24601138")):
        s = subprocess.run(["git", "-C", f"{REPOS}/scikit-fem", "show", f"{rev}:skfem/models/elasticity.py"], capture_output=True, text=True).stdout
        i = s.index("def lame_parameters"); j = s.index("\ndef ", i + 10); src = s[i:j]
        spec = lambda E, nu: (E * nu / ((1 + nu) * (1 - 2 * nu)), E / (2 * (1 + nu)))     # λ, μ (정의)
        g = {}; exec(src, g); fn = g["lame_parameters"]
        b0 = [(nu, not np.allclose(fn(1000.0, nu), spec(1000.0, nu))) for nu in (0.25, 0.3)]
        fe, _ = oc.exact_version(src, "lame_parameters", {}); rg = random.Random(2); E, nu = gf_rand(rg), gf_rand(rg)
        got, want = fe(E, nu), spec(E, nu)
        b2 = all(GF.c(a).v == GF.c(b).v for a, b in zip(got, want))
        # 같음 층: λ = μ  <=>  ν = 1/4  (E ≠ 0).  그 층에서도 판정
        lay = fe(E, Fr(1, 4)); lay_ok = all(GF.c(a).v == GF.c(b).v for a, b in zip(lay, spec(E, GF.c(Fr(1, 4)))))
        out[ver] = dict(B0=[f"ν={nu}: {'잡음' if d else '놓침'}" for nu, d in b0], B1="한 번의 실행(ν = 0.3)에서 정의와 비교: " + ("반례" if b0[1][1] else "통과"),
                        B2="증명" if b2 else "반례(무작위 E, ν)", B2_layer_nu_quarter="통과" if lay_ok else "반례")
    return out


# ---------------------------------------------------------------- R4 nodepy (대조군)
def r4():
    import sympy as sp
    out = {}
    for ver in ("bug", "fix"):
        sys.path.insert(0, f"{RB}/nodepy_{ver}")
        for k in [k for k in list(sys.modules) if k.startswith("nodepy")]: del sys.modules[k]
        import nodepy.runge_kutta_method as rkm
        RK = rkm.loadRKM()
        res = {}
        for name in ("SSP22", "SSP33"):
            m = RK[name]; s = len(m)
            # B2: 기호 실행 — 해석하지 않은 f, 기호 u, dt로 Shu-Osher 걸음 = 계수표(A, b)의 걸음
            f = oc.sym_f(1); u = np.array([sp.Symbol("u")], dtype=object); dt = sp.Symbol("h")
            try:
                y = m.__step__(lambda t, v: f(v), 0, u, dt, use_butcher=False)
                A = [[Fr(str(a)) for a in row] for row in m.A]; b = [Fr(str(x)) for x in m.b]
                ref = oc.rk_step(A, b, f, u, dt)
                step_ok = oc.canon(np.asarray(y).ravel()[0] - np.asarray(ref).ravel()[0]) == 0
            except Exception as e:
                step_ok = f"실행 실패 {type(e).__name__}"
            try:
                m.stability_function_unexpanded(); stab = "통과"
            except Exception as e:
                stab = f"예외 {type(e).__name__}"
            res[name] = dict(step=step_ok, stability_unexpanded=stab)
        out[ver] = res; sys.path.pop(0)
    return out


if __name__ == "__main__":
    R = dict(R1=r1(), R2=r2(), R3=r3(), R4=r4())
    json.dump(R, open("realbug_results.json", "w"), ensure_ascii=False, indent=1, default=str)
    print(json.dumps(R, ensure_ascii=False, indent=1, default=str))
```

작동 확인 spec_error_test.py — 22.5절: 명세 오류

```
"""23절: 명세가 틀리면? — 틀린 명세를 충실히 따르는 코드에 사슬 전체를 건다.

  사슬   방정식 --(정확도 층: verify, 1–12절)--> 출력
         주장한 성질(4차) --(공식 층: 코드에서 읽은 차수, 20.5절)--> 의도한 식
         의도한 식 --(구현 층: 닫힘, 20.2–20.4절)--> 코드
  틀린 명세: RK4 계수표의 그럴듯한 오타 하나, 낮은 차수 방법에 "4차"를 붙인 것.  맞는 명세: RK4, 3/8 규칙.
  코드는 각 명세의 계수를 그대로 적은 평범한 RK 걸음이다(코드 = 명세).
"""
import json, time, warnings
from fractions import Fraction as Fr
import numpy as np
warnings.filterwarnings("ignore")
import oracle_close as oc
from semidisc import Sys
from verify import verify

F = Fr
RK4 = ([[0, 0, 0, 0], [F(1, 2), 0, 0, 0], [0, F(1, 2), 0, 0], [0, 0, 1, 0]], [F(1, 6), F(1, 3), F(1, 3), F(1, 6)])
CORRECT = {
    "RK4": RK4,
    "3/8 규칙": ([[0, 0, 0, 0], [F(1, 3), 0, 0, 0], [F(-1, 3), 1, 0, 0], [1, -1, 1, 0]], [F(1, 8), F(3, 8), F(3, 8), F(1, 8)]),
}


def typos():
    """RK4 표의 0이 아닌 칸마다 그럴듯한 오타(1/2<->1/3, 1/3<->1/6, 1<->1/2), b의 두 칸 맞바꿈."""
    out = {}
    alt = {F(1, 2): [F(1, 3), F(1, 4)], F(1, 3): [F(1, 6), F(1, 2)], F(1, 6): [F(1, 3), F(1, 8)], F(1): [F(1, 2)]}
    A, b = RK4
    for i in range(4):
        for j in range(4):
            if A[i][j] != 0:
                for a in alt[F(A[i][j])]:
                    A2 = [r[:] for r in A]; A2[i][j] = a; out[f"a{i+1}{j+1}: {A[i][j]} -> {a}"] = (A2, b[:])
    for i in range(4):
        for a in alt[F(b[i])]:
            b2 = b[:]; b2[i] = a; out[f"b{i+1}: {b[i]} -> {a}"] = ([r[:] for r in A], b2)
    b2 = b[:]; b2[0], b2[1] = b2[1], b2[0]; out["b1 <-> b2"] = ([r[:] for r in A], b2)
    A2 = [r[:] for r in A]; A2[3][2], A2[3][1] = 0, 1; out["a43 <-> a42 (k3 자리에 k2)"] = (A2, b[:])
    return out


MISLABEL = {
    "호인 2차를 4차로": ([[0, 0], [1, 0]], [F(1, 2), F(1, 2)]),
    "SSPRK3를 4차로": ([[0, 0, 0], [1, 0, 0], [F(1, 4), F(1, 4), 0]], [F(1, 6), F(1, 6), F(2, 3)]),
    "쿠타 3차를 4차로": ([[0, 0, 0], [F(1, 2), 0, 0], [-1, 2, 0]], [F(1, 6), F(2, 3), F(1, 6)]),
    "중점법을 4차로": ([[0, 0], [F(1, 2), 0]], [0, 1]),
}


def code_for(A, b):
    """명세의 계수를 그대로 적은 평범한 코드(사용자가 표를 옮겨 적은 것)."""
    s = len(b); L = ["def rk_run(f, y0, h, n):", "    Y = [y0]", "    y = y0", "    for it in range(n):"]
    for i in range(s):
        arg = "y" + "".join(f" + h * ({A[i][j].numerator} / {A[i][j].denominator}) * k{j}" if isinstance(A[i][j], Fr) else f" + h * ({A[i][j]}) * k{j}"
                            for j in range(i) if A[i][j] != 0)
        L.append(f"        k{i} = f({arg})")
    comb = " + ".join(f"({F(b[i]).numerator} / {F(b[i]).denominator}) * k{i}" for i in range(s) if b[i] != 0)
    L += [f"        y = y + h * ({comb})", "        Y.append(y)", "    return Y"]
    return "\n".join(L) + "\n"


S = Sys(dict(m=3, N=1, L=np.zeros((1, 1)), w=np.ones(1), D=[0.0, 0.0, 0.0], T=1.0, u0=[0.1, 1.0, 0.0],
             R=lambda s: [s[0] * (1 - s[0]), -s[2], s[1]]))


def judge(name, A, b, claim=4):
    src = code_for(A, b)
    rel = lambda f, Y, h, k: oc.rk_step(A, b, f, Y[k], h)
    impl = oc.close_time(src, "rk_run", {}, rel)["verdict"]                       # 구현 층: 코드 = 명
```

```
    세?
    order, bad = oc.code_order(src, "rk_run", {}, pmax=5)                          # 공식 층: 코드의 차수
    fn = oc.float_version(src, "rk_run", {})
    step = lambda Ff, y, h: np.asarray(fn(Ff, np.asarray(y, float), h, 1)[-1], float)
    t0 = time.time()
    rep = verify(S, step=step, N=20, claim=None, use_library=False, eps=1e-6, name=name)   # 정확도 층: 방정식의 해와
    acc = rep["판정"]["status"]
    return dict(name=name, impl=impl, order=order, order_ok=order >= claim,
                first_bad=(oc.tree_str(bad[0][0]), str(bad[0][2]), str(bad[0][3])) if bad and order < claim else None,
                accuracy=acc, accuracy_text=rep["판정"]["text"][:120], sec=round(time.time() - t0, 1))


if __name__ == "__main__":
    out = []
    for group, specs in (("맞는 명세", CORRECT), ("오타", typos()), ("차수 오표기", MISLABEL)):
        for nm, (A, b) in specs.items():
            r = judge(nm, A, b); r["group"] = group; out.append(r); print(r, flush=True)
            json.dump(out, open("spec_error_results.json", "w"), ensure_ascii=False, indent=1)
```

작동 확인 lorenz_wall.py — 22.6절: 정확도 벽(자원)

```
"""정확도 벽이 자원으로 밀리는가: 로렌츠 T=20에서 기준해 자원(max_ref)과 걸음 수를 키운다(23절)."""
import sys, time, json, numpy as np, warnings
warnings.filterwarnings("ignore")
from semidisc import Sys
from verify import verify, library
from schemes_lib import rk4
RK4c = library()["RK4"]; out = []
for T, spu, mref in [(15.0, 400, 4096), (20.0, 400, 4096), (20.0, 800, 16384), (20.0, 1600, 65536), (25.0, 1600, 65536)]:
    S = Sys(dict(m=3, N=1, L=np.zeros((1, 1)), w=np.ones(1), D=[0.0, 0.0, 0.0], T=T, u0=[1.0, 1.0, 1.0],
                 R=lambda s: [10 * (s[1] - s[0]), s[0] * (28 - s[2]) - s[1], s[0] * s[1] - 8 * s[2] / 3]))
    t0 = time.time()
    try:
        rep = verify(S, step=rk4, N=int(spu * T), claim=RK4c, claim_name="RK4", eps=1e-3, name="로렌츠", max_ref=mref)
        r = dict(T=T, steps_per_unit=spu, max_ref=mref, verdict=rep["판정"]["status"], text=rep["판정"]["text"][:160],
                 when=rep.get("언제", {}).get("text", "")[:160], sec=round(time.time() - t0))
    except Exception as e:
        r = dict(T=T, steps_per_unit=spu, max_ref=mref, verdict="오류", text=f"{type(e).__name__}: {e}"[:160], sec=round(time.time() - t0))
    out.append(r); print(r, flush=True); json.dump(out, open("lorenz_wall.json", "w"), ensure_ascii=False, indent=1)
```

작동 확인 lorenz_wall2.py — 22.6절: 테일러 12차 기준해

```
"""23절(22.6): 테일러 차수(n_ref)를 올린 기준해로 로렌츠 T=20."""
import time, json, numpy as np, warnings
warnings.filterwarnings("ignore")
from semidisc import Sys
from verify import verify, library
from schemes_lib import rk4
RK4c = library()["RK4"]; out = []
for T, spu, mref, K in [(20.0, 400, 4096, 12), (20.0, 400, 8192, 16)]:
    S = Sys(dict(m=3, N=1, L=np.zeros((1, 1)), w=np.ones(1), D=[0.0, 0.0, 0.0], T=T, u0=[1.0, 1.0, 1.0],
                 R=lambda s: [10 * (s[1] - s[0]), s[0] * (28 - s[2]) - s[1], s[0] * s[1] - 8 * s[2] / 3]))
    t0 = time.time()
    rep = verify(S, step=rk4, N=int(spu * T), claim=RK4c, claim_name="RK4", eps=1e-3, name="로렌츠", max_ref=mref, n_ref=K)
    r = dict(T=T, steps_per_unit=spu, max_ref=mref, taylor=K, verdict=rep["판정"]["status"], text=rep["판정"]["text"][:160],
             when=rep.get("언제", {}).get("text", "")[:160], sec=round(time.time() - t0))
    out.append(r); print(r, flush=True); json.dump(out, open("lorenz_wall2.json", "w"), ensure_ascii=False, indent=1)
```

작동 확인 lorenz_wall3.py — 22.6절: 촘촘한 후보

```
"""22.6절: 같은 T=20, 테일러 12차 기준해로, 후보의 걸음을 4배 촘촘히(1600걸음/단위)."""
import time, json, numpy as np, warnings
warnings.filterwarnings("ignore")
from semidisc import Sys
from verify import verify, library
from schemes_lib import rk4
RK4c = library()["RK4"]
T, spu, mref, K = 20.0, 1600, 4096, 12
S = Sys(dict(m=3, N=1, L=np.zeros((1, 1)), w=np.ones(1), D=[0.0, 0.0, 0.0], T=T, u0=[1.0, 1.0, 1.0],
             R=lambda s: [10 * (s[1] - s[0]), s[0] * (28 - s[2]) - s[1], s[0] * s[1] - 8 * s[2] / 3]))
t0 = time.time()
rep = verify(S, step=rk4, N=int(spu * T), claim=RK4c, claim_name="RK4", eps=1e-3, name="로렌츠", max_ref
=mref, n_ref=K)
r = dict(T=T, steps_per_unit=spu, max_ref=mref, taylor=K, verdict=rep["판정"]["status"], text=rep["판정"
]["text"][:160],
         when=rep.get("언제", {}).get("text", "")[:160], sec=round(time.time() - t0))
print(r, flush=True); json.dump([r], open("lorenz_wall3.json", "w"), ensure_ascii=False, indent=1)
```

작동 확인 pysdc_case.py — 22.4절: pySDC 상태 누수

```
"""22.4절의 다섯째 실제 버그: pySDC c44d9c3f0(2026-09 수정) — 숨은 전역 상태(클래스 속성) 누수.
mode=fresh  : 닫힘 검사처럼 깨끗한 상태에서 Child의 명시적 계수표 = 의도(가중치 없으면 암시적 가중치)
mode=history: 사용자의 실행처럼 Parent 수준을 먼저 만든 뒤 같은 검사"""
import sys, json, numpy as np
ver, mode = sys.argv[1], sys.argv[2]
sys.path.insert(0, f"../realbugs/pysdc_{ver}")
from pySDC.implementations.sweeper_classes.Runge_Kutta import IMEXEuler
from pySDC.core.level import Level
from pySDC.implementations.problem_classes.TestEquation_0D import testequation0d


class Parent(IMEXEuler):
    weights_explicit = None


class Child(Parent):
    weights = np.array([0.5])


if mode == "history":
    Level(problem_class=testequation0d, problem_params={}, sweeper_class=Parent, sweeper_params={}, leve
l_params={"dt": 0.1}, level_index=0)
got = np.asarray(Child.get_Butcher_tableau_explicit().weights, float).ravel()
print(json.dumps(dict(ver=ver, mode=mode, got=got.tolist(), want=[0.5], ok=bool(np.allclose(got, [0.5]))
)))
```

nn_close.py — 22.8절 실험 1: 신경망 추론의 차원 무관 닫힘

```
"""22.8절 실험 1: 신경망 추론 구현의 오라클을 차원과 무관하게 닫는다.

ReLU 망의 활성 패턴은 2^(뉴런 수)개라 결정 열거(20.4절)는 폭이 크면 끝나지 않는다.  대신 코드가 부르는
활성 함수
(max(v, 0))를 해석하지 않은 함수 σ로 읽는다.  그러면 망은 선형 사상과 σ의 합성이 되고, GF 판정(20.2절)이
모든 입력·모든 가중치·모든 σ에서 "코드의 식 = 의도한 식"을 판정한다.  σ가 정말 ReLU인지는 따로 증명한다:
max(v, 0)과 relu는 v < 0, v = 0, v > 0 세 조각에서 각각 일차식이므로, 조각마다 두 점(v = 0은 한 점)에서
같으면
그 조각 전체에서 같다(정확한 유리수로).
"""
import json, random, time
from fractions import Fraction as Fr
import numpy as np
import oracle_close as oc
from oracle_close import GF, gf_rand, Tracer, TInt
import mutate

MLP_SRC = '''
def mlp_forward(x, Ws, bs):
    h = x
    for l in range(len(Ws)):
        W = Ws[l]
        b = bs[l]
        z = []
        for i in range(len(W)):
            s = b[i]
            for j in range(len(h)):
                s = s + W[i][j] * h[j]
```

```
            z.append(s)
        if l < len(Ws) - 1:
            h = [max(v, 0) for v in z]
        else:
            h = z
    return h
'''


class Sigma:
    """해석하지 않은 스칼라 함수(무작위 신탁)."""
    def __init__(s, rng): s.rng, s.tab = rng, {}
    def __call__(s, v):
        k = GF.c(v).v
        if k not in s.tab: s.tab[k] = gf_rand(s.rng)
        return s.tab[k]


def spec_forward(x, Ws, bs, sig):
    h = list(x)
    for l in range(len(Ws)):
        z = [bs[l][i] + sum((Ws[l][i][j] * h[j] for j in range(len(h))), GF(0)) for i in range(len(Ws[l]))]
        h = [sig(v) for v in z] if l < len(Ws) - 1 else z
    return h


class DList(list):
    """차원 꼬리표가 붙은 리스트: len이 이 꼬리표를 출처로 기록한다(깊이와 각 층의 폭은 서로 다른 차원).
    """
    def __init__(s, it, root): super().__init__(it); s.dimroot = root


def make_net(rng, dims):
    L = len(dims) - 1
    Ws = DList([DList([DList([gf_rand(rng) for _ in range(dims[l])], f"w{l}") for _ in range(dims[l + 1])], f"w{l + 1}") for l in range(L)], "depth")
    bs = DList([DList([gf_rand(rng) for _ in range(dims[l + 1])], f"w{l + 1}") for l in range(L)], "depth")
    return DList([gf_rand(rng) for _ in range(dims[0])], "w0"), Ws, bs


def run_once(fn, g, dims, seed, tr=None):
    rng = random.Random(seed); sig = Sigma(rng); x, Ws, bs = make_net(rng, dims)
    def gmax(a, b):
        if isinstance(b, int) and not isinstance(b, bool) and b == 0: return sig(a)       # max(v, 0) -> σ(v)
        if isinstance(a, int) and not isinstance(a, bool) and a == 0: return sig(b)
        raise oc.ValueBranch("max의 다른 쓰임")
    tr = tr or Tracer(); old = oc._patched(g, tr); g["max"] = gmax
    try:
        out = fn(x, Ws, bs)
    except oc.ValueBranch as e:
        return None, dict(dims=dims, why=f"범위 밖: {e}")
    except Exception as e:
        return False, dict(dims=dims, why=f"실행 실패: {type(e).__name__}")
    finally:
        oc._restore(g, old); g.pop("max", None)
    want = spec_forward(x, Ws, bs, sig)
    if len(out) != len(want): return False, dict(dims=dims, why="출력 길이")
    for i, (a, b) in enumerate(zip(out, want)):
        if GF.c(a).v != GF.c(b).v: return False, dict(dims=dims, why=f"출력 {i}")
    return True, dict(dims=dims)


def relu_primitive_ok(fn_max=max):
    """max(v, 0) = relu(v): 세 조각(v<0, v=0, v>0)에서 조각마다 두 점(정확한 유리수)."""
    relu = lambda v: v if v > 0 else 0
    pts = [Fr(-3), Fr(-1, 7), Fr(0), Fr(1, 9), Fr(5)]
    return all(fn_max(v, 0) == relu(v) for v in pts)


def judge(src, sizes_all=True):
    """정사각 층(모든 차원 n, 깊이 3)에서 모든 n(대표 크기, 20.3절) + 직사각 표본 + 큰 차원 한 번."""
    fn, g = oc.exact_version(src, "mlp_forward", {})
    import domains as Dm
    def run(n, seed, tr): return run_once(fn, g, [n, n, n, n], seed, tr)
    r = Dm.close_sized(run, 1)
    if r["verdict"] not in ("증명", "유한 증명"): return r
    for k, dims in enumerate([[3, 7, 5, 2], [10, 4, 9, 1], [1, 1, 6, 3], [17, 2, 2, 8]]):
```

```
        ok, info = run_once(fn, g, dims, 100 + k)
        if ok is None: return dict(verdict="범위 밖", info=info)
        if not ok: return dict(verdict="반례", info=info)
    return dict(verdict=r["verdict"], meta=r.get("meta"))


def float_test(src):
    """흔한 시험: 한 망(4-8-8-3)에서 무작위 부동소수 입력으로 numpy 참조 구현과 비교."""
    try:
        fn = oc.float_version(src, "mlp_forward", {}); rng = np.random.default_rng(0)
        dims = [4, 8, 8, 3]; Ws = [rng.standard_normal((dims[l + 1], dims[l])) for l in range(3)]; bs = [rng.standard_normal(dims[l + 1]) for l in range(3)]
        x = rng.standard_normal(4); h = x
        for l in range(3):
            h = Ws[l] @ h + bs[l]; h = np.maximum(h, 0) if l < 2 else h
        out = fn(x.tolist(), [W.tolist() for W in Ws], [b.tolist() for b in bs])
        return not np.allclose(out, h)
    except Exception:
        return True


if __name__ == "__main__":
    res = dict(relu_primitive=relu_primitive_ok())
    fn, g = oc.exact_version(MLP_SRC, "mlp_forward", {})
    scale = []
    for d, H in [(1, 4), (10, 32), (100, 100), (300, 300), (1000, 64)]:
        t0 = time.time(); ok, info = run_once(fn, g, [d, H, H, 3], 7); scale.append(dict(d=d, H=H, ok=ok, sec=round(time.time() - t0, 2),
                                                                                        patterns=f"2^{2 * H}")); print(scale[-1], flush=True)
    res["scale"] = scale
    t0 = time.time(); res["original"] = judge(MLP_SRC); res["original_sec"] = round(time.time() - t0, 1)
    ms = mutate.mutants(MLP_SRC); rows = []
    for kind, where, line, s in ms:
        b = float_test(s)
        try: r = judge(s)
        except Exception as e: r = dict(verdict="반례", info=f"판정 중 {type(e).__name__}")
        rows.append(dict(kind=kind, where=where, B2=r["verdict"], info=str(r.get("info", ""))[:80], BR=b, src=s))
    res["mutants"] = rows
    json.dump(res, open("nn_close_results.json", "w"), ensure_ascii=False, indent=1, default=str)
    import collections
    print(res["original"], collections.Counter((r["B2"], r["BR"]) for r in rows))
```

작동 확인 nn_oracle.py — 22.8절 실험 2: 학습된 오라클

```
"""22.8절 실험 2: 신경망이 빠진 명세를 대신할 수 있는가 — 학습된 오라클.

명세(의도한 식)를 주지 않고, "맞는 RK4 출력"과 "틀린 구현의 출력"의 예만으로 신경망을 학습시켜 맞고 틀림을 가르게 한다.
보편 근사 정리는 충분히 넓은 망이 어떤 판별 함수든 근사할 수 있다고 말한다.  여기서 재는 것은 그 근사가 처음 보는 버그와 처음 보는 입력에서 몇 번 틀리는가이다.
  자료   맞는 RK4(원본) 1개와 동치가 아닌 RK4 변이체(20.6절, 실행 실패 제외)
  입력   (y0, 10걸음의 궤적)  — 우변 f: 로지스틱 + 회전,  h = 0.1
  분할   변이체를 반씩 학습/시험(처음 보는 버그),  초기값: 학습 상자 [-1,1]^3, 시험은 같은 상자(분포 안)와 3배 상자(분포 밖)
  망     ReLU MLP, 폭 64·256·1024, 깊이 3
"""
import json, math, random, warnings
import numpy as np
import torch
warnings.filterwarnings("ignore")
import oracle_close as oc
import targets as T

torch.manual_seed(0); np.random.seed(0)
f = lambda y: np.array([y[0] * (1 - y[0]), -y[2], y[1]])
R = json.load(open("oracle_results.json"))
muts = [r["src"] for r in R if r["key"] == "rk4" and r["kind"] in ("AOR", "CRP", "ROR", "SVR", "IDX") and not r["crash"] and r["B2"][0] == "반례"]
orig = T.RK4
random.Random(1).shuffle(muts); tr_m, te_m = muts[: len(muts) // 2], muts[len(muts) // 2:]


def traj(src, y0):
    try:
        fn = oc.float_version(src, "rk4_run", {})
        with np.errstate(all="ignore"): Y = np.array([np.asarray(v, float) for v in fn(f, y0.copy(), 0.1, 10)])
```

```
        if Y.shape != (11, 3): return None
        return np.clip(np.nan_to_num(Y, nan=1e6, posinf=1e6, neginf=-1e6), -1e6, 1e6)
    except Exception:
        return None


def feats(y0, Y): return np.concatenate([y0, Y[1:].ravel()])


def dataset(progs, n_y0, scale, rng, label):
    X, y = [], []
    for p in progs:
        for _ in range(n_y0):
            y0 = rng.uniform(-scale, scale, 3); Y = traj(p, y0)
            if Y is not None: X.append(feats(y0, Y)); y.append(label)
    return X, y


def build(rng):
    Xp, yp = dataset([orig], 3000, 1.0, rng, 1)
    Xn, yn = dataset(tr_m, 35, 1.0, rng, 0)
    return np.array(Xp + Xn, np.float32), np.array(yp + yn, np.float32)


def train(X, y, width, epochs=400):
    mu, sd = X.mean(0), X.std(0) + 1e-9
    net = torch.nn.Sequential(torch.nn.Linear(X.shape[1], width), torch.nn.ReLU(), torch.nn.Linear(width, width), torch.nn.ReLU(),
                              torch.nn.Linear(width, 1))
    opt = torch.optim.Adam(net.parameters(), lr=1e-3); Xt = torch.tensor((X - mu) / sd); yt = torch.tensor(y)[:, None]
    w = torch.tensor([(y == 0).sum() / max(1, (y == 1).sum())], dtype=torch.float32)
    lossf = torch.nn.BCEWithLogitsLoss(pos_weight=w)
    for ep in range(epochs):
        perm = torch.randperm(len(Xt))
        for i in range(0, len(Xt), 512):
            idx = perm[i:i + 512]; opt.zero_grad(); l = lossf(net(Xt[idx]), yt[idx]); l.backward(); opt.step()
    return lambda Z: (torch.sigmoid(net(torch.tensor((np.asarray(Z, np.float32) - mu) / sd))).detach().numpy().ravel() > 0.5)


def evaluate(pred, rng, scale):
    # 맞는 프로그램을 틀렸다고(거짓 반례), 틀린 프로그램을 맞다고(거짓 증명) — 표본(입력) 단위와 프로그램 단위
    Xp, _ = dataset([orig], 400, scale, rng, 1); fr = 1 - pred(np.array(Xp, np.float32)).mean()
    fa_samples, fa_prog = [], 0
    for p in te_m:
        Xn, _ = dataset([p], 20, scale, rng, 0)
        if not Xn: continue
        acc = pred(np.array(Xn, np.float32)); fa_samples.append(acc.mean()); fa_prog += acc.mean() > 0.5
    return dict(false_reject=float(fr), false_accept_samples=float(np.mean(fa_samples)), false_accept_programs=int(fa_prog), test_programs=len(te_m))


if __name__ == "__main__":
    rng = np.random.default_rng(0); X, y = build(rng); out = dict(n_train=len(X), n_mut_train=len(tr_m), n_mut_test=len(te_m), runs=[])
    for width in (64, 256, 1024):
        pred = train(X, y, width)
        r = dict(width=width, in_dist=evaluate(pred, np.random.default_rng(10), 1.0), out_dist=evaluate(pred, np.random.default_rng(11), 3.0))
        out["runs"].append(r); print(r, flush=True)
    json.dump(out, open("nn_oracle_results.json", "w"), ensure_ascii=False, indent=1)
```

incomplete_oracle.py — 22.9절: 미완성 오라클의 육하원칙 보고

```
"""incomplete_oracle: 미완성 오라클이 왜 판정하지 못하는지를 육하원칙으로 낸다(22.9절).

오라클이 미완성인 경우는 넷이다. 넷 다 검증기가 스스로 짚는다.
  A 범위 밖        판정 장치가 다룰 수 없는 연산을 만났다(값의 대소, 초월 함수, 체에 없는 값, 명세 없음).
                   코드의 어느 줄에서 멈췄는지를 낸다.
  B 명세 공백      증명이 났지만 명세가 출력의 일부를 정하지 않는다.  증명 뒤에 출력의 길이·원소·순서를 하나씩
                   바꿔 보고, 바꿔도 명세가 참이면 그 부분이 공백이다.
  C 반올림 의존    정확 산술에서 0이 아닌 두 값의 뺄셈이 늘 정확히 0이 되는 줄이 있다.  그 코드의 의도는 반올림
                   수준에 있고(보정 합산 등), 정확 산술 판정은 그것을 보지 못한다.
```

```
  D 숨은 상태 의존 같은 판정을 깨끗한 상태와 앞선 호출 뒤에 했을 때 결과가 다르다.

보고의 여섯 칸: 누가(어느 오라클·명세), 무엇을(빠진 것), 언제(입력 크기·조건), 어디서(코드·출력의 자리),
왜(판정이 서지 않는 까닭), 어떻게(오라클을 완성하는 방법).
"""
import copy, random, sys, traceback
import numpy as np
import oracle_close as oc
from oracle_close import GF, gf_rand, ValueBranch

W6 = ("누가", "무엇을", "언제", "어디서", "왜", "어떻게")


def report(kind, **kw):
    r = dict(종류=kind); r.update({k: kw.get(k, "") for k in W6}); return r


def fmt(r):
    return f"[{r['종류']}] " + " | ".join(f"{k}: {r[k]}" for k in W6 if r[k])


# ---------------------------------------------------------------- A 범위 밖: 멈춘 자리
def code_site(exc, src):
    """예외의 추적에서 대상 코드(<exact ...> 또는 <float ...>)의 줄을 찾는다."""
    lines = [l for l in src.strip("\n").split("\n")]
    for fr, ln in reversed(list(traceback.walk_tb(exc.__traceback__))):
        if fr.f_code.co_filename.startswith("<exact") or fr.f_code.co_filename.startswith("<float"):
            i = ln - 1 - (1 if src.startswith("\n") else 0)
            txt = lines[i].strip() if 0 <= i < len(lines) else "?"
            return f"{ln - (1 if src.startswith(chr(10)) else 0)}번째 줄 `{txt}`"
    return "명세 쪽(대상 코드 밖)"


WHY_A = {
    "값에 대한 비교": ("값의 대소 비교(부호·경계·허용치)", "GF에는 순서가 없어 대소를 정할 수 없다",
                  "대수 부분은 결정 열거(20.4절)로, 대소가 뜻하는 성질은 실행 검사의 부등식(1-11절)으로 판정한다"),
    "값의 같음 비교": ("값의 같음 분기", "무작위 값에서는 같음이 거의 늘 거짓이라 같음 층을 덮지 못한다",
                 "같음 술어의 층마다 입력을 만들어 층별로 판정한다(21.3절)"),
    "GF -> float": ("체 원소를 실수로 바꾸는 연산(초월 함수 등)", "명세나 코드가 sin·exp 같은 초월 함수를 체 위에서 요구한다",
                    "명세를 주장한 다항식으로 바꾸거나, 알려진 특수값을 가로챈다(21.2절의 FFT 단위근)"),
    "체에 없는 단위근": ("체에 없는 단위근", "요구한 단위근의 차수가 이 체의 곱셈군에 없다",
                  "그 차수를 가진 소수로 체를 바꾸거나 실행 검사로 판정한다"),
    "max의 다른 쓰임": ("해석하지 않은 함수로 읽을 수 없는 활성 함수", "max(v, c)에서 c ≠ 0이라 σ(v)로 읽히지 않는다",
                   "그 함수를 σ로 등록하거나 결정 열거로 판정한다"),
    "abs": ("절댓값", "절댓값은 대수 연산이 아니다", "결정 열거에서 불투명 값으로 읽는다(20.4절)"),
}


def explain_out_of_scope(oracle, why, site, when):
    for key, (what, reason, how) in WHY_A.items():
        if key in why:
            return report("A 범위 밖", 누가=oracle, 무엇을=what, 언제=when, 어디서=site, 왜=reason, 어떻게=how)
    return report("A 범위 밖", 누가=oracle, 무엇을=why, 언제=when, 어디서=site, 왜="판정 장치가 다루지 않는 연산", 어떻게="실행 검사로 판정한다")


def explain_no_spec(oracle, target):
    return report("A 범위 밖", 누가=oracle, 무엇을="명세 자체", 언제="항상", 어디서=target,
                  왜="맞는 출력이 식·방정식·기댓값·분포로 정의되어 있지 않아 비교할 기준이 없다",
                  어떻게="맞는 출력을 수학적으로 정의한다(22.8절: 예로 학습한 신경망은 대신하지 못한다)")


def probe_A(fn, args_fn, spec, src, oracle, when):
    """GF로 한 번 돌려 범위 밖이면 육하원칙 보고(아니면 None)."""
    args = args_fn()
    try:
        out = fn(*args)
    except ValueBranch as e:
        return explain_out_of_scope(oracle, str(e), code_site(e, src), when)
    try:
        spec(args, out)
    except ValueBranch as e:
        return explain_out_of_scope(oracle, str(e), "명세 함수 안", when)
    return None
```

```
83
84
85  # ---------------------------------------------------------------- B 명세 공백: 출력을 흔들어 본다
86  def _fresh_like(x, rng):
87      if isinstance(x, GF): return gf_rand(rng)
88      if isinstance(x, (int, float, np.floating)) and not isinstance(x, bool): return type(x)(x) + 1 + rng
    .random()
89      if hasattr(x, "k"):                                  # 비교 전용 키
90          y = copy.copy(x); y.k = x.k + 1 + rng.randrange(5); return y
91      if isinstance(x, (bytes,)): return bytes(rng.randrange(256) for _ in range(len(x)))
92      return None
93
94
95  def _same(a, b):
96      if a is b: return True
97      try: return GF.c(a).v == GF.c(b).v
98      except Exception: pass
99      try: return bool(a == b) and type(a) == type(b) and not hasattr(a, "k")
100     except Exception: return False
101
102
103 def perturbations(out, rng, path="출력"):
104     """(설명, 바뀐 출력) 들. 목록·배열·튜플·사전을 따라 내려간다."""
105     if isinstance(out, tuple) and not isinstance(out, (str, bytes)):
106         for i, part in enumerate(out):
107             for d, p in perturbations(part, rng, f"{path}[{i}]"):
108                 yield d, out[:i] + (p,) + out[i + 1:]
109         return
110     if isinstance(out, dict):
111         ks = list(out)
112         for k in ks[:6]:
113             v = _fresh_like(out[k], rng)
114             if v is not None: o = dict(out); o[k] = v; yield (f"{path}['{k}'] 값", ("원소", f"{path}['{k
    }']")), o
115         o = dict(out); o["__새 키__"] = next(iter(out.values())); yield (f"{path}에 키 하나 더", ("길이"
    , path)), o
116         return
117     if isinstance(out, (list, np.ndarray)):
118         L = list(out); n = len(L)
119         if n:
120             yield (f"{path} 끝에 원소 하나 더", ("길이", path)), L + [copy.copy(L[-1])]
121             yield (f"{path} 마지막 원소 빼기", ("길이", path)), L[:-1]
122         for i in range(min(n, 8)):
123             v = _fresh_like(L[i], rng)
124             if v is not None: o = L[:]; o[i] = v; yield (f"{path}[{i}] 바꾸기", ("원소", f"{path}[{i}]")
    ), o
125         for i in range(min(n, 8)):
126             for j in range(i + 1, min(n, 8)):
127                 if not _same(L[i], L[j]) and (not hasattr(L[i], "k") or L[i].k == L[j].k) and (hasattr(L
    [i], "k") or not _equal_value(L[i], L[j])):
128                     o = L[:]; o[i], o[j] = o[j], o[i]; yield (f"{path}[{i}] ↔ [{j}]", ("순서", f"{path}[
    {i}] ↔ [{j}]" + (" (같은 키)" if hasattr(L[i], "k") else ""))), o
129         return
130
131
132 def _equal_value(a, b):
133     try: return GF.c(a).v == GF.c(b).v
134     except Exception:
135         try: return a == b
136         except Exception: return False
137
138
139 def probe_B(args, out, spec, oracle, when, seed=0):
140     """증명이 난 출력에서 명세 공백을 찾는다. 공백마다 육하원칙 보고."""
141     rng = random.Random(seed); gaps = {}; passing = {}
142     for (desc, (cat, where)), p in perturbations(out, rng):
143         try:
144             ok = spec(args, p)[0]
145         except Exception:
146             ok = False
147         if ok: gaps.setdefault(cat, []).append(where); passing.setdefault(cat, []).append(p)
148     rep = []
149     HOW = {"길이": "출력의 길이를 정하는 조건을 명세에 넣는다(예: len(out) == n)",
150            "원소": "그 원소를 정하는 식을 명세에 넣는다",
151            "순서": "같은 값(같은 키) 원소의 순서를 정하는 조건을 명세에 넣는다(예: 안정성 — 입력에서의
    순서 유지)"}
152     WHAT = {"길이": "출력의 길이", "원소": "출력의 원소 값", "순서": "출력 원소의 순서"}
153     for cat, wh in gaps.items():
154         r = report("B 명세 공백", 누가=oracle, 무엇을=WHAT[cat], 언제=when, 어디서=", ".join(sorted(set(
    wh))[:4]) + (" 외" if len(set(wh)) > 4 else ""),
```

```
                    왜=f"그 자리를 바꿔도 명세가 참으로 남는다({len(wh)}가지 바꿈)", 어떻게=HOW[cat])
        r["정답 후보"], r["지금 코드"] = candidates(cat, args, out, passing.get(cat, []))
        rep.append(r)
    total = sum(1 for _ in perturbations(out, random.Random(seed)))
    if gaps and sum(len(v) for v in gaps.values()) == total:
        rep = [report("B 명세 공백", 누가=oracle, 무엇을="출력 전체", 언제=when, 어디서="출력의 모든 자
리",
                      왜="어떻게 바꿔도 명세가 참이다: 명세가 출력을 전혀 정하지 않는다(명세 없음과 같다
)",
                      어떻게="맞는 출력을 수학적으로 정의한다")]
    return rep


def _len_of(o):
    if isinstance(o, tuple): return tuple(_len_of(x) for x in o)
    try: return len(o)
    except Exception: return None


def candidates(cat, args, out, alts):
    """명세의 빈칸이 허용하는 정답 후보들과, 지금 코드가 그중 무엇을 골랐는지.  후보 가운데 하나를 고르
는 기준은 명세 밖에 있다."""
    if cat == "길이":
        L0 = _len_of(out); Ls = sorted({str(_len_of(a)) for a in alts})
        return (f"길이 {L0}(지금 코드) 또는 늘린 길이 {', '.join(Ls)}(시험함) — 늘어난 자리는 명세가 보
지 않으므로 아무 값", f"길이 {L0}")
    if cat == "순서":
        items = list(out) if isinstance(out, (list, tuple)) else []
        if items and all(hasattr(t, "k") for t in items) and args and isinstance(args[0], (list, tuple))
:
            pos = {id(t): i for i, t in enumerate(args[0])}
            stable = all(not (items[i].k == items[j].k and pos.get(id(items[i]), 0) > pos.get(id(items[j
]), 0))
                         for i in range(len(items)) for j in range(i + 1, len(items)))
            groups = {}
            for t in items: groups[t.k] = groups.get(t.k, 0) + 1
            import math as _m
            n_orders = _m.prod(_m.factorial(c) for c in groups.values())
            return (f"같은 키 원소를 배치하는 {n_orders}가지 순서 가운데 아무것이나: ① 입력 순서 유지(안
정) ② 그 밖의 {n_orders - 1}가지",
                    "① 입력 순서 유지(안정)" if stable else "② 입력 순서와 다름(불안정)")
        return (f"지금 순서와 바꾼 순서 {len(alts)}가지", "지금 순서")
    if cat == "원소":
        return (f"그 자리의 값이 무엇이든({len(alts)}가지 바꿈 모두 통과)", "지금 값")
    return ("", "")


def fmt_full(r):
    s = fmt(r)
    if r.get("정답 후보"): s += f" | 정답 후보: {r['정답 후보']} | 지금 코드: {r['지금 코드']}"
    return s


# ---------------------------------------------------------------- C 반올림 의존: 정확히 0이 되는 뺄셈
class ZeroWatch:
    """뺄셈 연산 하나(줄 + 바이트코드 위치)마다, 0이 아닌 두 값의 뺄셈이 몇 번 일어났고 그중 몇 번이 정
확히 0이었는지."""
    def __init__(s, src): s.tot, s.zero = {}, {}
    def __enter__(s):
        s.orig = GF.__sub__
        def sub(a, b, _w=s):
            r = s.orig(a, b)
            if r is NotImplemented: return r
            bb = GF.c(b)
            if a.v != 0 and bb is not NotImplemented and bb.v != 0:
                f = sys._getframe(1)
                while f and not f.f_code.co_filename.startswith("<exact"): f = f.f_back
                if f:
                    key = (f.f_lineno - 1, f.f_lasti)
                    _w.tot[key] = _w.tot.get(key, 0) + 1
                    if r.v == 0: _w.zero[key] = _w.zero.get(key, 0) + 1
            return r
        GF.__sub__ = sub; return s
    def __exit__(s, *a): GF.__sub__ = s.orig; return False


def probe_C(fn, args_fn, src, oracle, when, runs=3):
    """실행될 때마다 정확히 0이 되는 뺄셈(0이 아닌 두 값)만 짚는다.  마지막 반복에서만 0이 되는 것
(결레기올기법의 정확 종료처럼 알고리즘의 정상 성질)은 짚지 않는다."""
    lines = src.strip("\n").split("\n"); tot, zero = {}, {}
```

```
    for _ in range(runs):
        with ZeroWatch(src) as w:
            try: fn(*args_fn())
            except Exception: pass
        for k, v in w.tot.items(): tot[k] = tot.get(k, 0) + v
        for k, v in w.zero.items(): zero[k] = zero.get(k, 0) + v
    always = [k for k in tot if tot[k] >= 2 and zero.get(k, 0) == tot[k]]
    if not always: return None
    k = max(always, key=lambda k_: tot[k_]); ln = k[0]; txt = lines[ln - 1].strip() if 0 < ln <= len(lin
es) else "?"
    return report("C 반올림 의존", 누가=oracle, 무엇을="반올림을 되살리는 보정 항", 언제=f"{when}, 무작
위 입력 {runs}벌에서 실행 {tot[k]}번 모두",
                  어디서=f"{ln}번째 줄 `{txt}`", 왜="0이 아닌 두 값의 뺄셈이 정확 산술에서 실행될 때마다
 0이다: 이 줄의 뜻은 반올림 오차에만 있고 정확 산술 판정은 그것을 보지 못한다",
                  어떻게="반올림 수준의 명세(예: |오차| <= 2u·Σ|x|)를 세우고 11절의 반올림 엄밀성으로 실
행마다 판정한다")


# ---------------------------------------------------------------- D 숨은 상태 의존
def probe_D(check_fresh, check_history, oracle, when, history_desc):
    a, b = check_fresh(), check_history()
    if a == b: return None
    return report("D 숨은 상태 의존", 누가=oracle, 무엇을="앞선 호출이 남긴 상태", 언제=f"{when}; {histo
ry_desc}",
                  어디서=f"깨끗한 상태의 판정 {a} ≠ 앞선 호출 뒤의 판정 {b}",
                  왜="출력이 입력만이 아니라 숨은 상태의 함수이다: 모든 입력에서의 판정은 그 상태를 입력
으로 보지 않는다",
                  어떻게="상태를 명시적 입력으로 꺼내거나, 실행 검사로 실제 호출 순서 그대로 판정한다")
```

작동 확인 incomplete_test.py — 22.9절의 실험

```
"""22.9절 실험: 미완성 오라클의 육하원칙 보고 — 실험에서 본 미완성 사례마다 검증기가 스스로 원인을 짚는
가,
그리고 완성된 오라클에서는 아무것도 짚지 않는가(헛짚음 0)."""
import json, random, subprocess
import numpy as np
import oracle_close as oc
from oracle_close import GF, gf_rand
import incomplete_oracle as IO
import domains as Dm, targets as T

R = []


def add(case, expect, reps):
    reps = [r for r in reps if r]
    R.append(dict(case=case, expect=expect, reports=reps, hit=(len(reps) > 0) == (expect != "없음") and
all(r["종류"].startswith(expect[0]) for r in reps) if expect != "없음" else len(reps) == 0))
    print(case, "->", [IO.fmt_full(r) for r in reps] or "보고 없음", flush=True)


def gf_args(make, n, seed=3):
    return lambda: make(random.Random(seed), n)


# ---- A 범위 밖
d = Dm.D["segint"]; fn, g = oc.exact_version(d["src"], d["fname"], {})
add("선분 교차(부호 명세)", "A", [IO.probe_A(fn, gf_args(d["make"], 1), d["spec"], d["src"], "닫힘 검사(
GF)", "모든 입력")])
d = Dm.D["sin_true"]; fn, g = oc.exact_version(d["src"], d["fname"], {})
add("sin 근사, 명세 = 참 sin", "A", [IO.probe_A(fn, gf_args(d["make"], 1), d["spec"], d["src"], "닫힘 검
사(GF)", "모든 x")])
d = Dm.D["mcpi"]; fn, g = oc.exact_version(d["src"], d["fname"], {})
rA = IO.probe_A(fn, gf_args(d["make"], 5), d["spec"], d["src"], "닫힘 검사(GF)", "점 5개")
ff = oc.float_version(d["src"], d["fname"], {}); pts = np.random.default_rng(0).random((50, 2)).tolist()
rB = IO.probe_B((pts,), ff(pts), d["spec"], "몬테카를로 π의 명세(참이면 통과)", "점 50개")
add("몬테카를로 π, 명세 없음과 같은 명세", "A", [rA] + rB)
fft_mut = Dm.D["fft"]["src"].replace("even = fft(x[0::2])", "even = fft(x[0::3])")
with oc.field(Dm.NTT_P):
    fn, g = oc.exact_version(fft_mut, "fft", dict(cmath=Dm._CM))
    add("FFT 보폭 변이체(x[0::3])", "A", [IO.probe_A(fn, gf_args(Dm.D["fft"]["make"], 4), Dm.D["fft"]["s
pec"], fft_mut, "닫힘 검사(GF, NTT 체)", "길이 16")])
add("학습된 모델의 정확도", "A", [IO.explain_no_spec("오라클 전체", "학습된 분류기의 예측")])

# ---- B 명세 공백
def lsq_spec_old(a, out):                       # 출력 길이 조건이 빠진 처음 명세(21.3절)
    A, b = a; m, n = len(A), len(A[0])
    res = [sum((A[k][j] * out[j] for j in range(n)), GF(0)) - b[k] for k in range(m)]
    return all(GF.c(sum((A[k][i] * res[k] for k in range(m)), GF(0))).v == 0 for i in range(n)), ""
```

```
fn, g = oc.exact_version(Dm.LSQ_SRC, "lstsq", {}); args = Dm.D["lstsq"]["make"](random.Random(1), 3); out = fn(*args)
add("최소제곱, 길이 조건 없는 명세", "B", IO.probe_B(args, out, lsq_spec_old, "최소제곱 명세(처음 판)", "n = 3"))
add("최소제곱, 길이 조건 넣은 명세(완성)", "없음", IO.probe_B(args, out, Dm.lsq_spec, "최소제곱 명세", "n = 3"))


class Key:
    def __init__(s, k, tag): s.k, s.tag = k, tag
    def __lt__(s, o): return s.k < o.k
    def __gt__(s, o): return s.k > o.k
    def __le__(s, o): return s.k <= o.k
    def __ge__(s, o): return s.k >= o.k
    def __repr__(s): return f"{s.k}{s.tag}"


fs = oc.float_version(Dm.D["isort"]["src"], "insertion_sort", {})
keys = [Key(k, t) for k, t in zip([2, 1, 2, 0, 1, 2], "abcdef")]; out = fs(keys)
spec_vals = lambda a, o: ([t.k for t in o] == sorted(t.k for t in a[0]), "")
spec_stable = lambda a, o: (o == sorted(a[0], key=lambda t: t.k), "")      # 같은 키는 입력 순서 그대로(안정 정렬)
add("삽입 정렬, 값만 보는 명세", "B", IO.probe_B((keys,), out, spec_vals, "정렬 명세(값)", "중복 키가 있는 6개"))
add("삽입 정렬, 안정성 넣은 명세(완성)", "없음", IO.probe_B((keys,), out, spec_stable, "정렬 명세(값 + 안정성)", "중복 키가 있는 6개"))

# ---- C 반올림 의존
d = Dm.D["kahan"]; fn, g = oc.exact_version(d["src"], d["fname"], {})
add("카한 합", "C", [IO.probe_C(fn, gf_args(d["make"], 8), d["src"], "닫힘 검사(GF)", "길이 8")])

# ---- D 숨은 상태 의존
def pysdc(mode, ver="bug"):
    o = subprocess.run(["python3", "pysdc_case.py", ver, mode], capture_output=True, text=True, timeout=300)
    line = (o.stdout.strip().splitlines() or [""])[-1]
    if line.startswith("{"): j = json.loads(line); return f"가중치 {j['got']}"
    return "예외 " + ((o.stderr.strip().splitlines() or ["?"])[-1].split(":")[0].split(".")[-1])
add("pySDC 상태 누수(고치기 전)", "D", [IO.probe_D(lambda: pysdc("fresh"), lambda: pysdc("history"), "닫힘 검사와 실행 검사", "자식 클래스의 명시적 계수표", "부모 수준을 먼저 만든 뒤")])
add("pySDC(고친 뒤, 완성)", "없음", [IO.probe_D(lambda: pysdc("fresh", "fix"), lambda: pysdc("history", "fix"), "닫힘 검사와 실행 검사", "자식 클래스의 명시적 계수표", "부모 수준을 먼저 만든 뒤")])

# ---- 완성된 오라클: 아무것도 짚지 않아야(헛짚음 검사)
rel = oc.intended_onestep("RK4")[0]
fn, g = oc.exact_version(T.RK4, "rk4_run", {})
def rk_make(): rng = random.Random(5); f = oc.Oracle(rng, 3); return (f, np.array([gf_rand(rng) for _ in range(3)], dtype=object), gf_rand(rng), 6)
a = rk_make(); Y = fn(*a)
def rk_spec(a_, o):
    f, y0, h, n = a_
    if len(o) != n + 1 or not oc.gf_eq(o[0], y0)[0]: return False, ""
    return all(oc.gf_eq(o[k + 1], rel(f, o, h, k))[0] for k in range(n)), ""
add("RK4(완성)", "없음", IO.probe_B(a, list(Y), rk_spec, "RK4 관계식", "n = 6") + [IO.probe_C(fn, rk_make, T.RK4, "닫힘 검사", "n = 6"),
     IO.probe_A(fn, rk_make, rk_spec, T.RK4, "닫힘 검사", "n = 6")])
fn, g = oc.exact_version(T.FEM, "fem_p1_apply", {})
def fem_make():
    rng = random.Random(8); x = np.array([gf_rand(rng) for _ in range(7)], dtype=object); return ([gf_rand(rng) for _ in range(7)], x)
a = fem_make(); out = fn(*a)
fem_spec = lambda a_, o: (len(o) == len(a_[0]) and all(GF.c(p).v == GF.c(q).v for p, q in zip(o, oc.graph_apply("FEM P1", list(a_[1]), a_[0]))), "")
add("FEM P1(완성)", "없음", IO.probe_B(a, out, fem_spec, "그래프 식", "N = 7") + [IO.probe_C(fn, fem_make, T.FEM, "닫힘 검사", "N = 7")])
for k in ("matmul", "conv", "bareiss", "cg"):
    d = Dm.D[k]; fn, g = oc.exact_version(d["src"], d["fname"], {}); a = d["make"](random.Random(2), 4); out = fn(*a)
    add(f"{d['label']}(완성)", "없음", IO.probe_B(a, out, d["spec"], d["label"], "n = 4") + [IO.probe_C(fn, gf_args(d["make"], 4), d["src"], "닫힘 검사", "n = 4")])
d = Dm.D["ecadd"]; fe = oc.float_version(d["src"], d["fname"], d["env"]); a = Dm.ec_make(random.Random(4), 0)
add("secp256k1 덧셈(완성)", "없음", IO.probe_B(a, fe(*a), lambda a_, o: (o == Dm.ec_ref(*a_), ""), "SEC 1 군 법칙", "일반 위치"))

json.dump(R, open("incomplete_results.json", "w"), ensure_ascii=False, indent=1)
print("맞게 짚음:", sum(r["hit"] for r in R), "/", len(R))
```

작동 확인 eqpin_test.py — 22.10절: 같음 갈래의 경로 조건 대입

```
"""22.10절 실험: 같음 갈래의 경로 조건 대입.
  실험 1  대조군 18개(버그·함정·무해·모순·비선형) — 고치기 전(같음 갈래를 무작위 점에서 판정)과 고친 뒤(경로 조건 대입)
  실험 2  변이 분석 — 같음 분기가 있는 RK4 대상 6개의 모든 변이체에서, 고친 뒤 잡는 힘이 줄었는가
반례마다 고정한 입력(경로 조건의 해)을 함께 적는다."""
import json, signal, difflib, warnings
import numpy as np
warnings.filterwarnings("ignore")
import oracle_close as oc, targets as T, mutate

L = "        k2 = f(y + h / 2 * k1)\n"
def ins(b): return T.RK4.replace(L, L + b)


def judge(src, pin, D=6):
    rel = oc.intended_onestep("RK4")[0]
    def check(out, f, args):
        for k in range(len(out) - 1):
            if not oc.gf_eq(out[k + 1], rel(f, out, args[1], k))[0]: return False, dict(step=k)
        return True, {}
    def al(*a): raise TimeoutError
    signal.signal(signal.SIGALRM, al); signal.alarm(10); oc.PIN[0] = pin
    try:
        fe, _ = oc.exact_version(src, "rk4_run", {})
        return oc.close_branching("def go(f, y0, h):\n    return rk4_run(f, y0, h, 2)\n", "go", {"rk4_run": fe}, check, d=3, D=D,
            inputs=lambda rng: (np.array([oc.gf_rand(rng) for _ in range(3)], dtype=object), oc.gf_rand(rng)))
    except TimeoutError: return dict(verdict="시간 초과")
    except Exception as e: return dict(verdict="반례", info=dict(why=f"출력 검사 실패 {type(e).__name__}"))
    finally: signal.alarm(0); oc.PIN[0] = True


CONTROLS = [  # (이름, 넣은 줄, 실제 정답, 정답의 이유)
 ("원본 RK4", None, "증명", "버그 없음"),
 ("h == 0.1이면 h/3 (처음 만든 버그)", "        if h == 0.1:\n            k2 = f(y + h / 3 * k1)\n", "반례", "h = 0.1에서 틀린다"),
 ("h == 0.1이면 h/2", "        if h == 0.1:\n            k2 = f(y + h / 2 * k1)\n", "증명", "같은 식"),
 ("h == 0.1이면 0.05", "        if h == 0.1:\n            k2 = f(y + 0.05 * k1)\n", "증명", "그 h에서 0.05 = h/2"),
 ("h != 0.1의 else에서 0.05", "        if h != 0.1:\n            pass\n        else:\n            k2 = f(y + 0.05 * k1)\n", "증명", "위와 같음"),
 ("h != 0.1의 else에서 0.04", "        if h != 0.1:\n            pass\n        else:\n            k2 = f(y + 0.04 * k1)\n", "반례", "0.04 ≠ h/2"),
 ("2h == y0[0]이면 y0[0]/4", "        if 2 * h == y0[0]:\n            k2 = f(y + y0[0] / 4 * k1)\n", "증명", "그 점에서 y0[0]/4 = h/2"),
 ("2h == y0[0]이면 y0[0]/2", "        if 2 * h == y0[0]:\n            k2 = f(y + y0[0] / 2 * k1)\n", "반례", "y0[0]/2 = h"),
 ("h == h이면 h/3", "        if h == h:\n            k2 = f(y + h / 3 * k1)\n", "반례", "늘 참"),
 ("h == h + 1이면 h/3", "        if h == h + 1:\n            k2 = f(y + h / 3 * k1)\n", "증명", "늘 거짓(죽은 코드)"),
 ("h == 0.1 안에 또 h == 0.1, 0.05", "        if h == 0.1:\n            if h == 0.1:\n                k2 = f(y + 0.05 * k1)\n", "증명", "겹친 조건"),
 ("h == 0.1 안에 h == 0.2이면 h/3", "        if h == 0.1:\n            if h == 0.2:\n                k2 = f(y + h / 3 * k1)\n", "증명", "모순 경로(죽은 코드)"),
 ("f의 값 k1[0] == 0이면 + k1[0]·k1", "        if k1[0] == 0:\n            k2 = f(y + h / 2 * k1 + k1[0] * k1)\n", "증명", "더한 항이 0"),
 ("f의 값 k1[0] == 0.5이면 h/3", "        if k1[0] == 0.5:\n            k2 = f(y + h / 3 * k1)\n", "반례", "그 f에서 틀린다"),
 ("2/h == y0[0]이면 1/y0[0]", "        if 2 / h == y0[0]:\n            k2 = f(y + 1 / y0[0] * k1)\n", "증명", "그 점에서 1/y0[0] = h/2(조건은 h에 비선형, y0[0]에 1차)"),
 ("2/h == y0[0]이면 2/y0[0]", "        if 2 / h == y0[0]:\n            k2 = f(y + 2 / y0[0] * k1)\n", "반례", "2/y0[0] = h. 판정점: 무작위 h, y0[0] := 2/h"),
 ("h·h == 0.01이면 0.05", "        if h * h == 0.01:\n            k2 = f(y + 0.05 * k1)\n", "범위 밖", "조건이 모든 변수에 비선형: 풀지 않는다"),
 ("h·h == 0.01의 else에서 h/3", "        if h * h == 0.01:\n            k2 = f(y + 0.05 * k1)\n        else:\n            k2 = f(y + h / 3 * k1)\n", "반례", "else 갈래(일반 위치)에서 틀린다"),
]
BASES = {"원본 RK4": T.RK4,
         "h == 0.1이면 h/3": ins(CONTROLS[1][1]),
         "h == 0.1이면 0.05": ins(CONTROLS[3][1]),
         "2h == y0[0]이면 y0[0]/4": ins(CONTROLS[6][1]),
         "k1[0] == 0이면 + k1[0]·k1": ins(CONTROLS[12][1]),
         "2/h == y0[0]이면 1/y0[0]": ins(CONTROLS[14][1])}

if __name__ == "__main__":
    out = dict(controls=[], mutants=[])
```

```
    for name, body, want, why in CONTROLS:
        src = T.RK4 if body is None else ins(body)
        o, n = judge(src, False), judge(src, True)
        pins = n.get("info", {}).get("pins", "") if isinstance(n.get("info"), dict) else ""
        out["controls"].append(dict(name=name, want=want, why=why, old=o["verdict"], new=n["verdict"], p
ins=pins))
        print(name, want, o["verdict"], n["verdict"], pins, flush=True)
    for bn, base in BASES.items():
        trans, lost = {}, []
        M = list(mutate.mutants(base))
        for m in M:
            src = m[-1]; o, n = judge(src, False)["verdict"], judge(src, True)["verdict"]
            trans[f"{o}→{n}"] = trans.get(f"{o}→{n}", 0) + 1
            if o == "반례" and n != "반례":
                d = [l[1:].strip() for l in difflib.unified_diff(base.splitlines(), src.splitlines(), li
neterm="", n=0) if l[:1] in "+-" and l[:3] not in ("---", "+++") and l[1:].strip()]
                lost.append(dict(now=n, before=d[0] if d else "", after=d[1] if len(d) > 1 else ""))
        out["mutants"].append(dict(base=bn, n=len(M), trans=trans, lost=lost)); print(bn, len(M), trans,
 flush=True)
    json.dump(out, open("eqpin_results.json", "w"), ensure_ascii=False, indent=1)
```

제6부 분야별 오라클 정의와 사용자 도구로 전부 닫기

오라클 문제를 해결했다. 정답 후보를 계산할 수 있는 문제는 정의 25.1로 닫는다: 자료와 맞는 정답 후보를 모두 내고, 그 후보는 더 줄일 수 없으며, 출력이 후보 밖이면 반례, 후보가 출력 하나뿐이면 증명이다. 계산할 수 없는 문제(정지 문제류)는 정의 25.2로 좁혀 가는 정답 후보 목록을 낸다. 모든 판정은 육하원칙으로 보고하고, 판정의 기준은 사용자의 도구 — 나머지 사다리, π의 연분수, 증폭, 그래프 상수, 반올림 부등식, 비율 편차, 육하원칙 — 에서 나온다.

25 오라클 문제의 정의와 판정 원칙

제5부 21절은 모든 분야를 소프트웨어 시험의 오라클 정의("출력이 의도한 식대로인가")로 판정했고, 판정 장치의 뼈대는 유한체 정확 재실행이었다. 이 부는 두 가지를 바꾸어 다시 닫는다.

- **분야별 정의.** 분야마다 그 분야의 실무자가 '맞는 출력'이라고 부르는 것을 명세로 쓴다. 블록체인에서는 체인 밖 값의 신뢰, ML에서는 정답 표지가 없는 예측이 오라클 문제이다.
- **사용자 도구.** 판정 기준은 나머지 사다리, π의 연분수, 증폭(i), 그래프 상수 (V, T), 반올림 부등식(11절), 큰수의 법칙·비율 편차, 육하원칙으로 한다. 검산식이 이미 완전한 분야는 그 검산식으로 판정하고 그렇다고 적는다.

정의 25.1 (오라클 문제와 그 해결). 입력 x, 출력 y, 자료 D(명세, 관측, 정답 표지 가운데 있는 것), 가정 A가 주어진다. 오라클은 **정답 후보 집합** $C(x, D, A)$를 내고 다음을 판정한다.

- **건전.** A가 성립하면 참답은 C 안에 있다(확률 명세면 확률 $1-\delta$ 이상).
- **최소.** C의 모든 원소 c마다, 관측 D와 가정 A를 그대로 만족하면서 참답이 c인 세계가 있다. 그래서 어떤 오라클도 C를 더 줄일 수 없다.
- **판정.** $C=\{y\}$이면 출력은 **맞다(증명)**. $y \notin C$이면 출력은 **틀리다(반례**, 증인은 그 입력). $y \in C$이고 C에 다른 원소가 있으면 '출력이 맞다'는 주장은 **반례**이고 증인은 참답이 다른 세계이다. 이때 '어떻게' 칸에 C를 줄일 자료를 적는다.
- **가정.** A 가운데 자료로 검사할 수 있는 것은 검사하고(경보는 δ-반례), 검사할 수 없는 것은 그 사실을 세계 쌍으로 보인다.
- **보고.** 모든 판정을 육하원칙으로 낸다.

오라클은 이 다섯을 모두 만족할 때 **해결**되었다고 한다.